Fortress Deploy

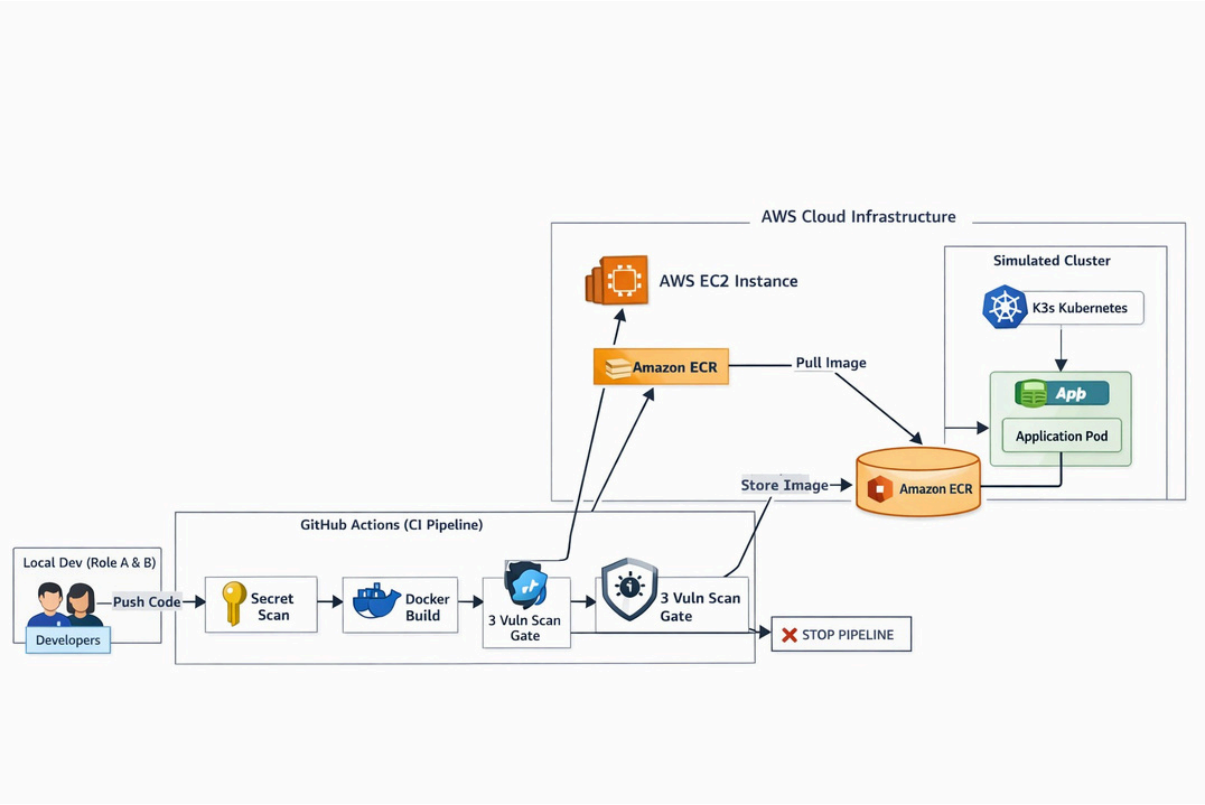
Fortress Deploy is a cloud-native DevSecOps project that demonstrates how to secure a CI/CD pipeline by enforcing automated security checks before application deployment.

The project simulates real-world scenarios such as hardcoded secrets and vulnerable dependencies to validate secret detection, container vulnerability scanning, and controlled Kubernetes deployments on AWS.

Key Highlights

- Secure CI/CD pipeline using GitHub Actions
- Automated secret detection and vulnerability scanning
- Dockerized Python Flask application
- Kubernetes deployment using K3s on AWS EC2
- Feature-branch workflow with protected main branch

Combined DevSecOps Workflow (Full Architecture Flow)



Project Status: Planning Phase **Target Platform:** AWS Cloud (Cloud-Native) **Architecture:** GitHub Actions → ECR → Kubernetes**1. The Problem Statement (Why are we building this?)**

The Scenario: We are simulating a critical situation in a Fintech or Healthcare startup. The development team is currently deploying code manually, which has led to serious security risks:

1. **Secret Leakage:** Developers have accidentally committed AWS keys and passwords to GitHub.
2. **Vulnerable Code:** A recent deployment contained a critical "Zero-Day" vulnerability (CVE) in a library, exposing the production environment to hackers.
3. **No Audit Trail:** There is no record of who deployed what or if the code was tested.

The Mission: Our goal is to build a **"Secure Supply Chain"** pipeline. We will enforce a "Zero Trust" policy where no code reaches production unless it passes automated security gates.

What We Are Solving:

- Eliminating human error in deployments.
- Automating the detection of security secrets (passwords/keys).
- Blocking vulnerabilities *before* the build is finished.

2. Project Architecture & Flow

We will implement a modern, Cloud-Native architecture used by top tech companies.

The Workflow:

1. **Development:** Developers push code to a **Centralized GitHub Repository**.
2. **Continuous Integration (CI):**
   - **Gate 1:** Code is scanned for secrets (API Keys, Passwords).
   - **Gate 2:** Docker Image is built.
   - **Gate 3 (The Security Wall):** The image is scanned by **Trivy**. If a *Critical* vulnerability is found, the pipeline **FAILS** immediately.
3. **Continuous Deployment (CD):**
   - If (and only if) the scan is clean, the image is pushed to **AWS ECR** (Private Registry).
   - The pipeline connects to the **Kubernetes Cluster** and updates the live application.

3. Roles & Responsibilities (Division of Labor)

To work efficiently and simulate a real DevOps team, we will divide the project

into two distinct domains: Infrastructure/App and Pipeline/Security.

Role A: Application & Infrastructure Lead

Focus: "The Foundation" (What are we deploying?) Responsibilities:

- Application Development: Write the Python/Node.js web application.
  - *Task:* Intentionally create a "vulnerable" version for testing purposes.
- Containerization: Write the Dockerfile to package the app.
- Kubernetes Infrastructure:
  - Provision the AWS EC2 Server.
  - Install and configure the Kubernetes Cluster (K3s).
  - Write the Kubernetes Manifests (deployment.yaml, service.yaml).

Role B: DevSecOps & Automation Lead

Focus: "The Pipeline" (How do we deploy it securely?) Responsibilities:

- Cloud Security Setup:
  - Create the AWS ECR (Elastic Container Registry).
  - Manage IAM Users and Permissions (ensuring GitHub has access).
- Pipeline Architecture:
  - Write the GitHub Actions Workflow (pipeline.yml).
  - Configure GitHub Secrets (API Keys, Credentials).
- Security Integration:
  - Implement the Trivy security scanner.
  - Define the "Failure Rules" (e.g., Block build on High Severity).

4. Collaboration Strategy (How We Work Together)

Since we are working on different parts of the same project, we will use a Centralized Repository workflow to avoid conflicts.

The "Git" Rules:

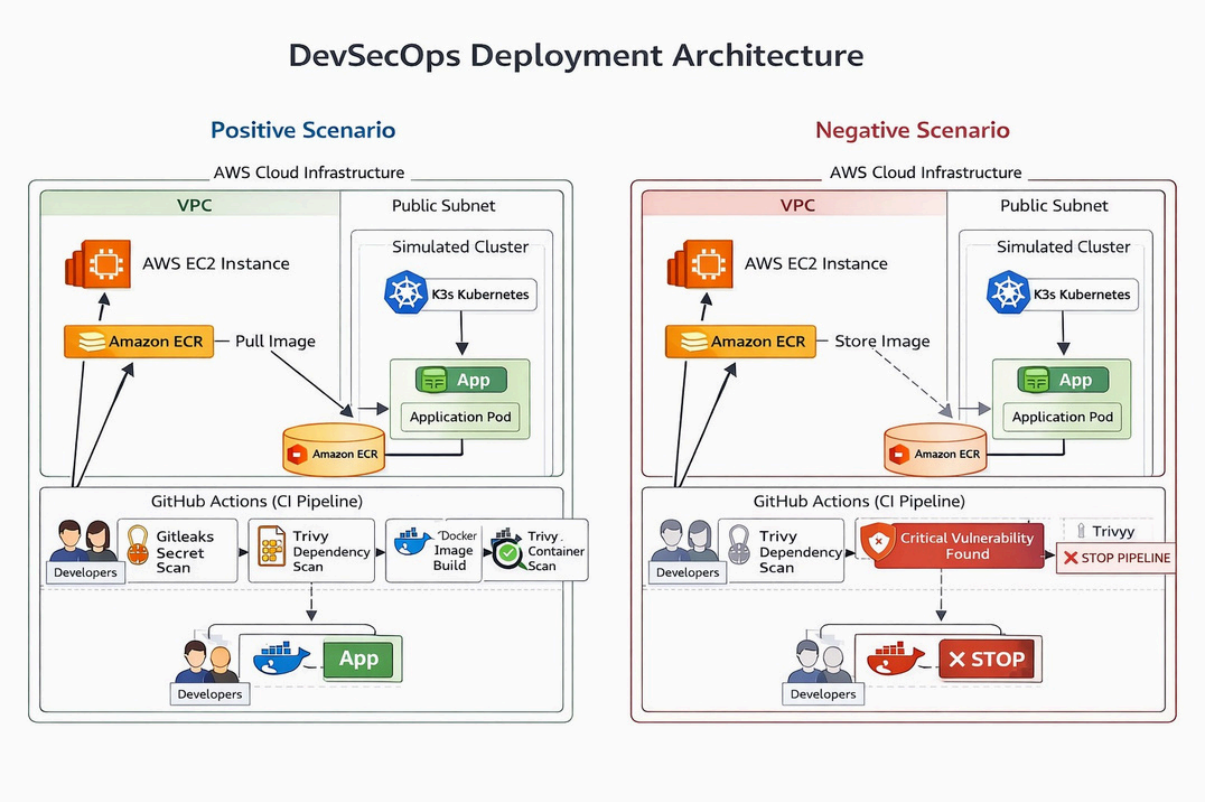
1. One Repo: We will use a single GitHub repository.
2. Two Branches:
   - Partner A works on branch: feature/app-infra
   - Partner B works on branch: feature/pipeline-security
3. No Direct Pushes to Main: We will never push directly to the main branch. We work in our feature branches and only merge when our part is working.
4. The "Integration Event":
   - Once Partner A has the App ready and Partner B has the Pipeline ready, we will merge both branches into main.

- This will trigger the First Build Run.

5. Definition of Done (Success Criteria)

The project is considered "Complete" when:

1. A developer pushes code with a vulnerability -> The Pipeline Fails (Red).
2. A developer fixes the vulnerability and pushes again -> The Pipeline Passes (Green).
3. The "Green" code is automatically running on the Kubernetes server without any manual human command.



Docker file -

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app/app.py .

# Create non-root user for better security
RUN useradd -m appuser
USER appuser

EXPOSE 5000

CMD ["python3", "app.py"]
```

Kubernetes Files

Deployment.yaml -

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fortress-deployment
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  selector:
    matchLabels:
      app: fortress
  template:
    metadata:
      labels:
        app: fortress
    spec:
      containers:
      - name: fortress-container
        image: fortress-project:1.0
```

```
        # Always pull latest image during deployment
        imagePullPolicy: Always
        ports:
        - containerPort: 5000

# This probe checks if the application is READY to receive traffic.
# Kubernetes will only send traffic to the pod if this probe succeeds.
# If the readiness probe fails, the pod will be temporarily removed
# from the service load balancer until it becomes healthy again.
  readinessProbe:
          httpGet:
            path: /health
            port: 5000
          initialDelaySeconds: 5
          periodSeconds: 5

# Liveness Probe
# ----------------
# This probe checks if the application is still running correctly.
# If the liveness probe fails repeatedly, Kubernetes will automatically
# restart the container to recover from a failure or deadlock state.
  livenessProbe:
    httpGet:
```

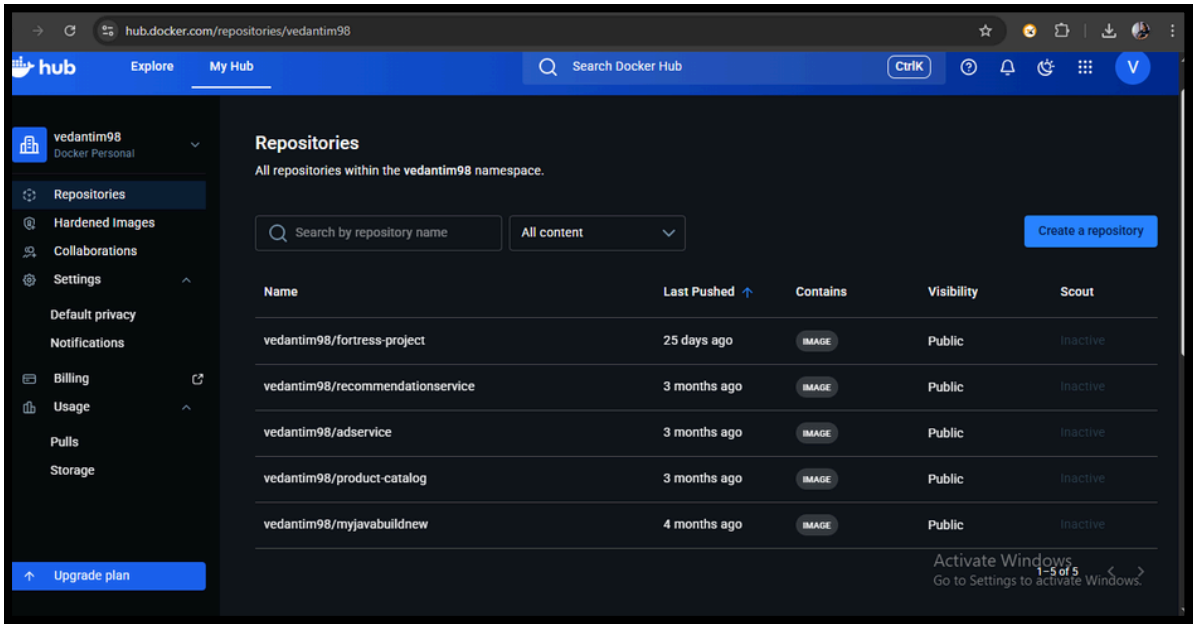
```
      path: /health
      port: 5000
    initialDelaySeconds: 10
    periodSeconds: 10
```

Service.yaml -

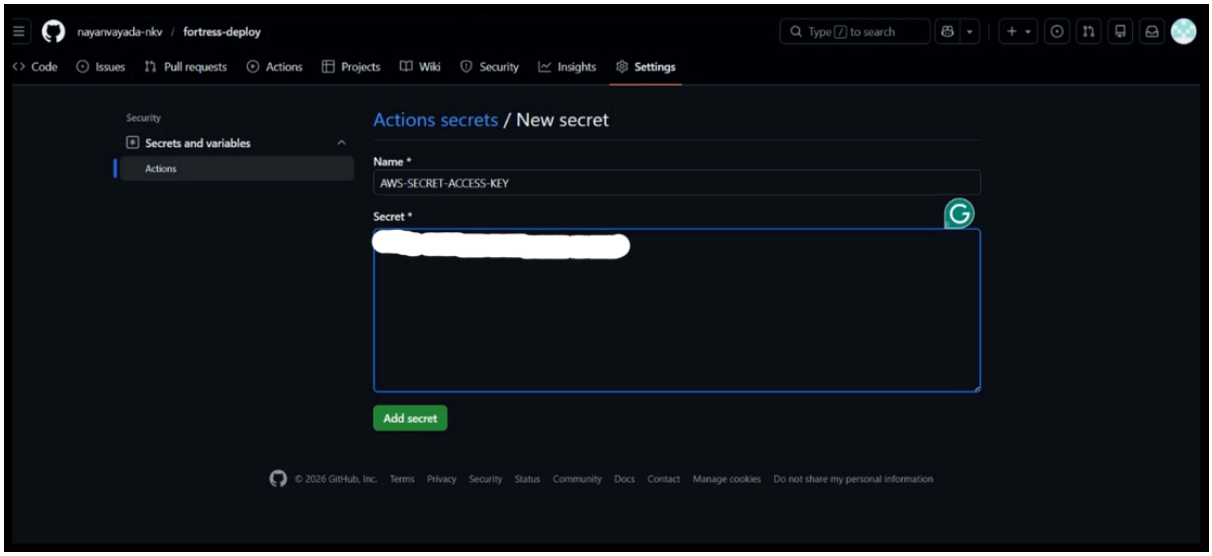
```
apiVersion: v1
kind: Service
metadata:
  name: fortress-service
spec:
  type: NodePort
  selector:
    app: fortress
  ports:
  - name: http
    protocol: TCP
    port: 5000
    targetPort: 5000
    nodePort: 30007
```

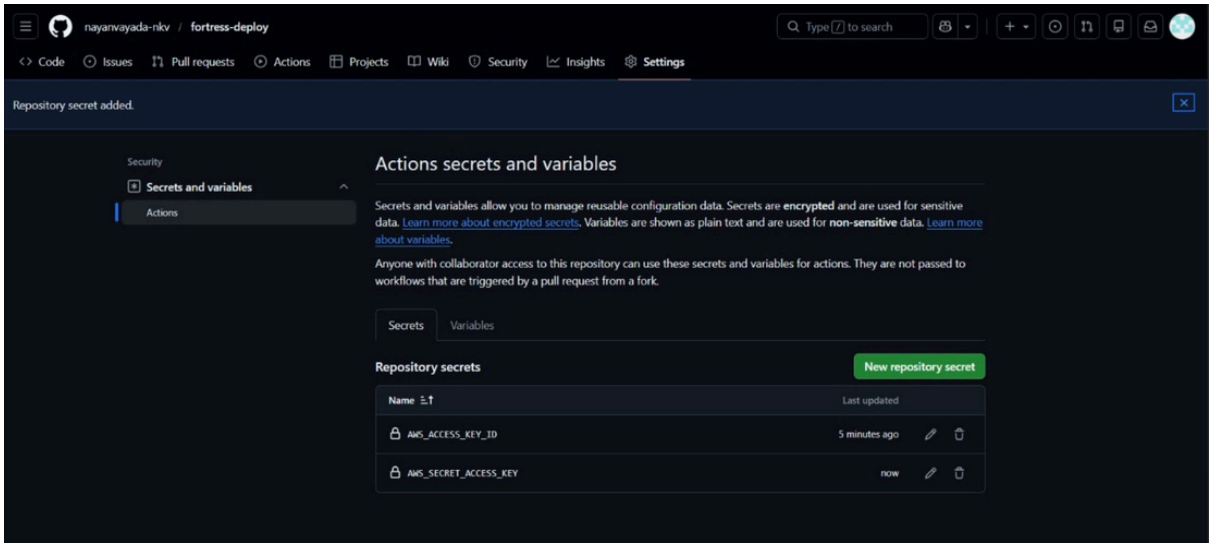
Locally project run by nodeport service -

Image stored in Docker Hub

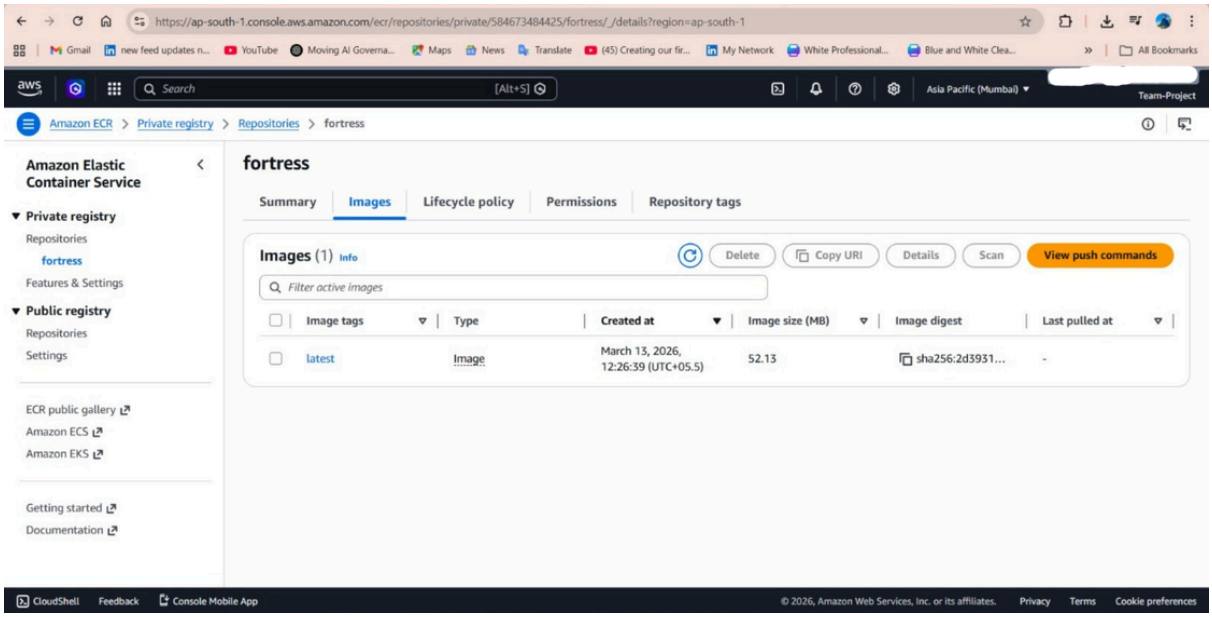


Added AWS Credentials in GitHub Access key and Secret access key

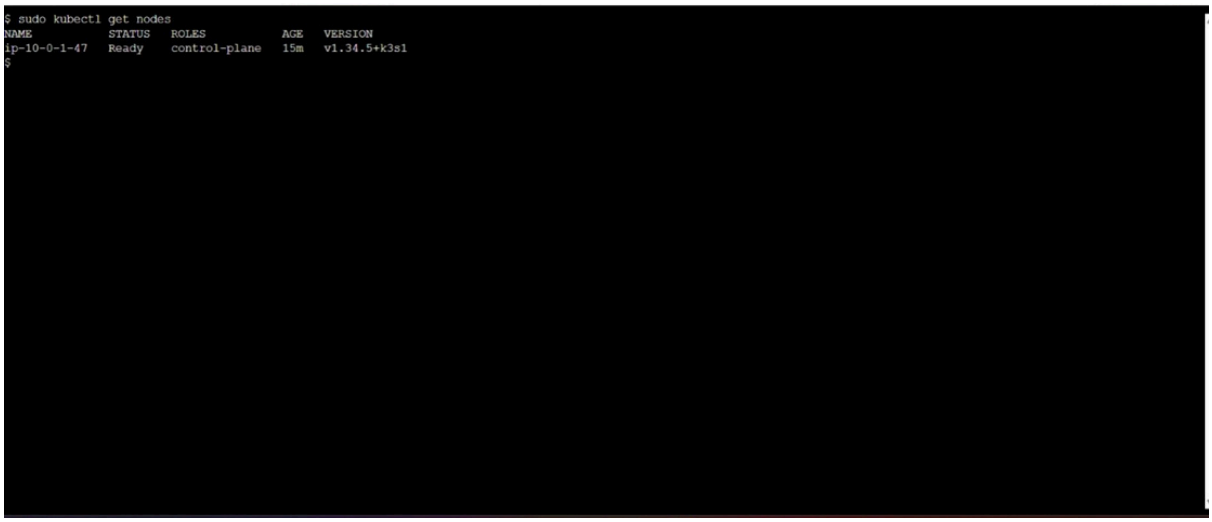




Push Image to Amazon ECR -



Kubernetes cluster - Running -



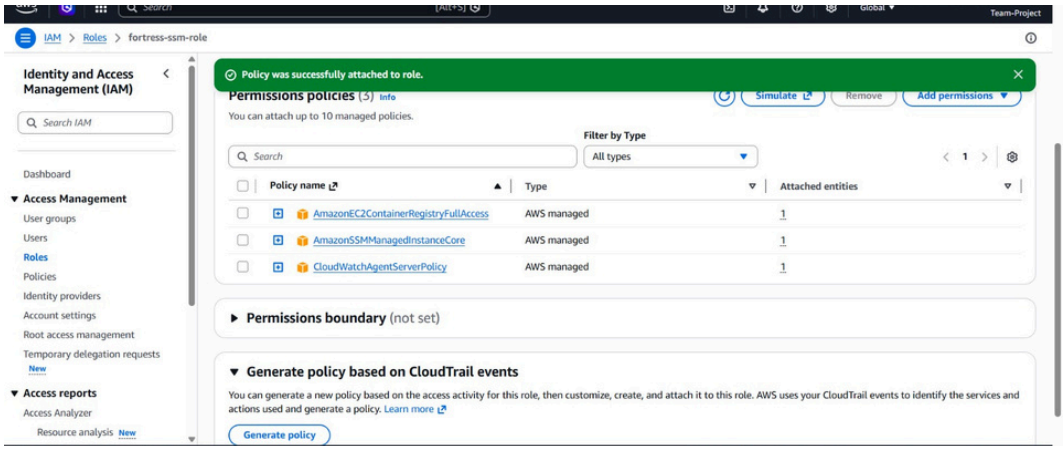
AWS CLI - Installation -

```
  inflating: aws/dist/awscli/topics/ddb-expressions.rst
  inflating: aws/dist/awscli/topics/topic-tags.json
  inflating: aws/dist/awscli/topics/s3-faq.rst
  inflating: aws/dist/awscli/topics/s3-case-insensitivity.rst
   creating: aws/dist/awscli/customizations/sso/
   creating: aws/dist/awscli/customizations/wizard/
   creating: aws/dist/awscli/customizations/wizard/wizards/
   creating: aws/dist/awscli/customizations/wizard/wizards/configure/
   creating: aws/dist/awscli/customizations/wizard/wizards/dynamodb/
   creating: aws/dist/awscli/customizations/wizard/wizards/events/
   creating: aws/dist/awscli/customizations/wizard/wizards/iam/
   creating: aws/dist/awscli/customizations/wizard/wizards/lambda/
  inflating: aws/dist/awscli/customizations/wizard/wizards/dynamodb/new-table.yml
  inflating: aws/dist/awscli/customizations/wizard/wizards/iam/new-role.yml
  inflating: aws/dist/awscli/customizations/wizard/wizards/lambda/new-function.yml
  inflating: aws/dist/awscli/customizations/wizard/wizards/configure/_main.yml
  inflating: aws/dist/awscli/customizations/wizard/wizards/events/new-rule.yml
  inflating: aws/dist/awscli/customizations/sso/index.html
  inflating: aws/dist/awscli/data/ac.index
  inflating: aws/dist/awscli/data/metadata.json
  inflating: aws/dist/awscli/data/cli.json
   creating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/
  inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/RECORD
  inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/INSTALLER
  inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/WHEEL
  inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/METADATA
  inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/top_level.txt
  inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/LICENSE
  inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/AUTHORS.rst
$ sudo ./aws/install
You can now run: /usr/local/bin/aws --version
$ aws --version
aws-cli/2.34.9 Python/3.13.11 Linux/6.17.0-1007-aws exe/x86_64.ubuntu.24
$
```

Docker - Installation -

```
Created symlink /etc/systemd/system/sockets.target.wants/docker.socket → /usr/lib/systemd/system/docker.socket.
Processing triggers for dbus (1.14.10-4ubuntu4.1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning candidates...
Scanning linux images...

Pending kernel upgrade!
Running kernel version:
  6.17.0-1007-aws
Diagnostics:
  The currently running kernel version is not the expected kernel version 6.17.0-1009-aws.

Restarting the system to load the new kernel will not be handled automatically, so you should consider rebooting.

Restarting services...

Service restarts being deferred:
 systemctl restart getty@tty1.service
 systemctl restart networkd-dispatcher.service
 systemctl restart serial-getty@ttyS0.service
 systemctl restart systemd-logind.service
 systemctl restart unattended-upgrades.service

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
$ sudo systemctl enable docker
sudo systemctl start dockersudo systemctl start docker$
$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
```

AmazonEC2ContainerRegistryFullAccess. Added permission to the SSM role for pulling container registry from ECR -



Login success -

```
Restarting the system to load the new kernel will not be handled automatically, so you should consider rebooting.

Restarting services...

Service restarts being deferred:
 systemctl restart getty@tty1.service
 systemctl restart networkd-dispatcher.service
 systemctl restart serial-getty@ttyS0.service
 systemctl restart systemd-logind.service
 systemctl restart unattended-upgrades.service

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
$ sudo systemctl enable docker
sudo systemctl start dockersudo systemctl start docker$
$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
$ aws ecr get-login-password --region ap-south-1 | sudo docker login --username AWS --password-stdin 584673484425.dkr.ecr.ap-south-1.amazonaws.com

aws: [ERROR]: An error occurred (AccessDeniedException) when calling the GetAuthorizationToken operation: User: arn:aws:sts::584673484425:assumed-role/fortress-ssm-role/i-0
e305e2e7ca092129 is not authorized to perform: ecr:GetAuthorizationToken on resource: * because no identity-based policy allows the ecr:GetAuthorizationToken action
Error: Cannot perform an interactive login from a non TTY device
$ aws ecr get-login-password --region ap-south-1 | sudo docker login --username AWS --password-stdin 584673484425.dkr.ecr.ap-south-1.amazonaws.com

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
```

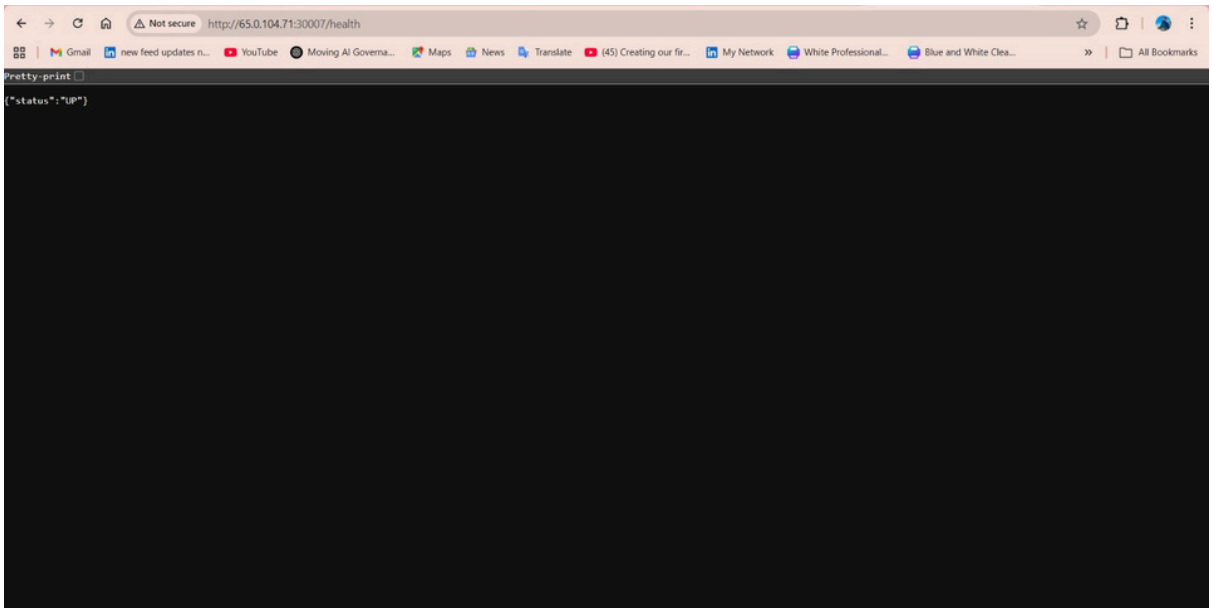
ECR secret created -

```
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
$ aws ecr get-login-password --region ap-south-1 | sudo docker login --username AWS --password-stdin 584673484425.dkr.ecr.ap-south-1.amazonaws.com

aws: [ERROR]: An error occurred (AccessDeniedException) when calling the GetAuthorizationToken operation: User: arn:aws:sts::584673484425:assumed-role/fortress-ssm-role/i-0
e305e2e7ca092129 is not authorized to perform: ecr:GetAuthorizationToken on resource: * because no identity-based policy allows the ecr:GetAuthorizationToken action
Error: Cannot perform an interactive login from a non TTY device
$ aws ecr get-login-password --region ap-south-1 | sudo docker login --username AWS --password-stdin 584673484425.dkr.ecr.ap-south-1.amazonaws.com

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
$ sudo kubectl delete pod --all
pod "fortress-999878d4f-b4bxj" deleted from default namespace
pod "fortress-999878d4f-pfvxj" deleted from default namespace
$ sudo kubectl get pods
NAME                        READY   STATUS             RESTARTS   AGE
fortress-999878d4f-4cczz    0/1     ImagePullBackOff   0          15s
fortress-999878d4f-s7s5m    0/1     ImagePullBackOff   0          15s
$ sudo kubectl get deployment fortress -o yaml | grep image
      {"apiVersion":"apps/v1","kind":"Deployment","metadata":{"annotations":{},"name":"fortress","namespace":"default"},"spec":{"replicas":2,"selector":{"matchLabels":{"app
":"fortress"}},"template":{"metadata":{"labels":{"app":"fortress"}},"spec":{"containers":[{"image":"584673484425.dkr.ecr.ap-south-1.amazonaws.com/fortress:latest","liveness
Probe":{"httpGet":{"path":"/health","port":5000},"initialDelaySeconds":10,"periodSeconds":20},"name":"fortress","ports":[{"containerPort":5000}],"readinessProbe":{"httpGet"
:{"path":"/health","port":5000},"initialDelaySeconds":5,"periodSeconds":10}}}}}}
      - image: 584673484425.dkr.ecr.ap-south-1.amazonaws.com/fortress:latest
        imagePullPolicy: Always
$ sudo kubectl create secret docker-registry ecr-secret \
--docker-server=584673484425.dkr.ecr.ap-south-1.amazonaws.com \
--docker-username=AWS \
--docker-password=$(aws ecr get-login-password --region ap-south-1) \
--docker-email=nayanvayada7@gmail.com
secret/ecr-secret created
```

Fortress pods running -

```
      {"apiVersion":"apps/v1","kind":"Deployment","metadata":{"annotations":{},"name":"fortress","namespace":"default"},"spec":{"replicas":2,"selector":{"matchLabels":{"app
":"fortress"}},"template":{"metadata":{"labels":{"app":"fortress"}},"spec":{"containers":[{"image":"584673484425.dkr.ecr.ap-south-1.amazonaws.com/fortress:latest","liveness
Probe":{"httpGet":{"path":"/health","port":5000},"initialDelaySeconds":10,"periodSeconds":20},"name":"fortress","ports":[{"containerPort":5000}],"readinessProbe":{"httpGet"
:{"path":"/health","port":5000},"initialDelaySeconds":5,"periodSeconds":10}}}}}}
      - image: 584673484425.dkr.ecr.ap-south-1.amazonaws.com/fortress:latest
        imagePullPolicy: Always
$ sudo kubectl create secret docker-registry ecr-secret \
--docker-server=584673484425.dkr.ecr.ap-south-1.amazonaws.com \
--docker-username=AWS \
--docker-password=$(aws ecr get-login-password --region ap-south-1) \
--docker-email=nayanvayada7@gmail.com
secret/ecr-secret created
$ nano k8s/deployment.yaml
$ cd ~/fortress-deploy
$ ls
README.md  app  docker  k8s  requirements.txt
$ nano k8s/deployment.yaml
$ sudo kubectl apply -f k8s/deployment.yaml
deployment.apps/fortress configured
$ sudo kubectl delete pod --all
pod "fortress-597596f9b6-25w99" deleted from default namespace
pod "fortress-597596f9b6-tl7h6" deleted from default namespace
pod "fortress-999878d4f-4cczz" deleted from default namespace
pod "fortress-999878d4f-s7s5m" deleted from default namespace
$ sudo kubectl get pods
NAME                        READY   STATUS        RESTARTS   AGE
fortress-597596f9b6-lfslb   1/1     Running       0          35s
fortress-597596f9b6-qtggs   1/1     Running       0          35s
fortress-999878d4f-flq54    1/1     Terminating   0          35s
$ sudo kubectl get pods
NAME                        READY   STATUS    RESTARTS   AGE
fortress-597596f9b6-lfslb   1/1     Running   0          88s
fortress-597596f9b6-qtggs   1/1     Running   0          88s
$
```

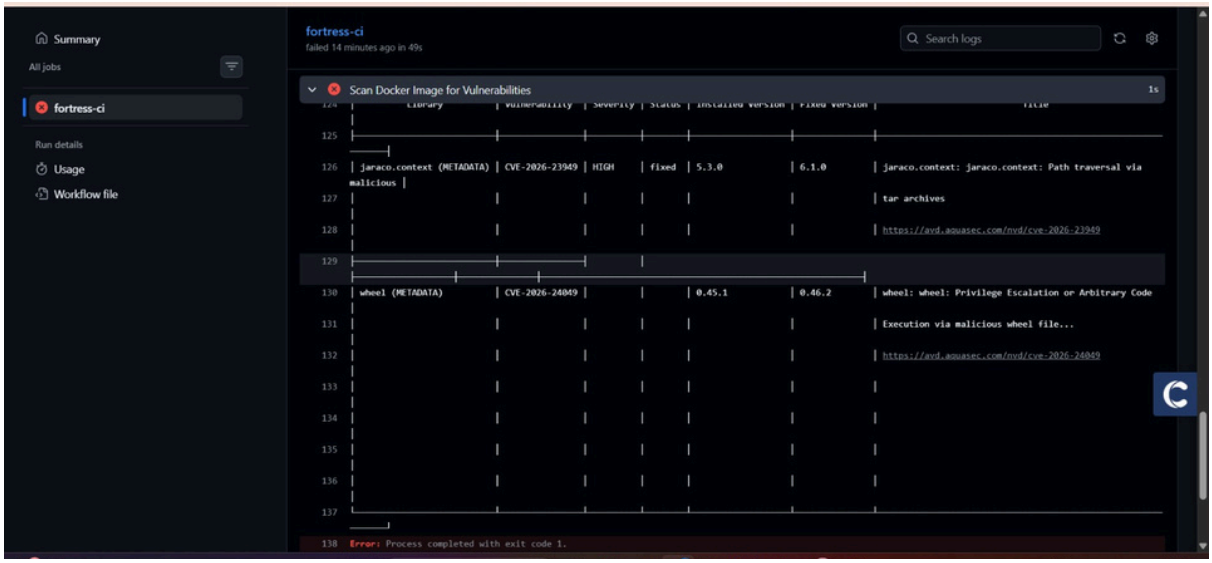
The application deployed on Kubernetes is successfully running and returning "status": "UP".-



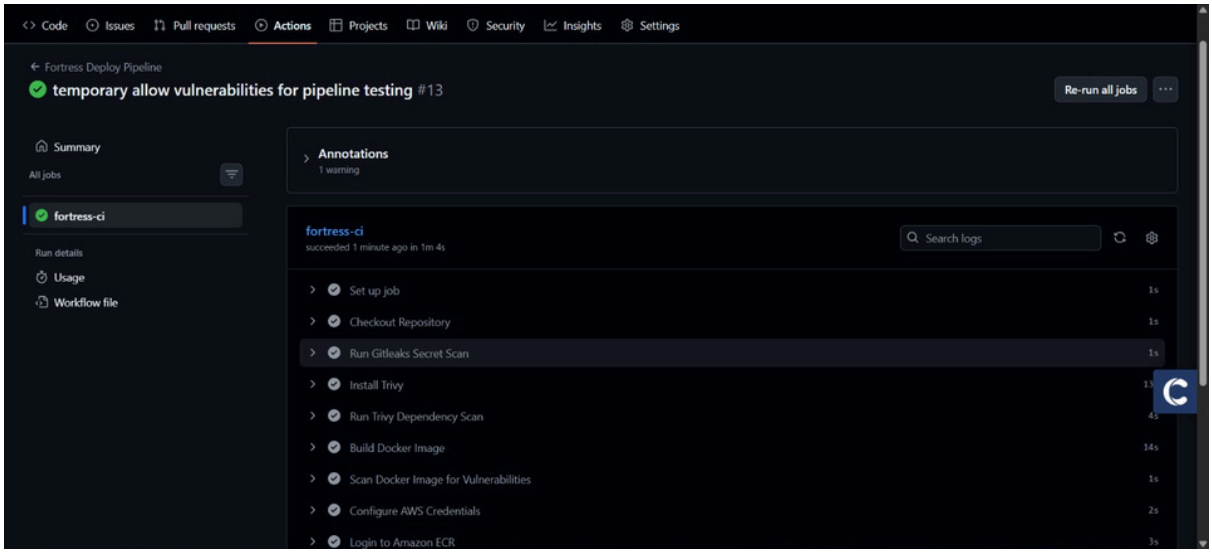
Devops-Testing

1. Test Trivy Image Scan (Critical Vulnerabilities)

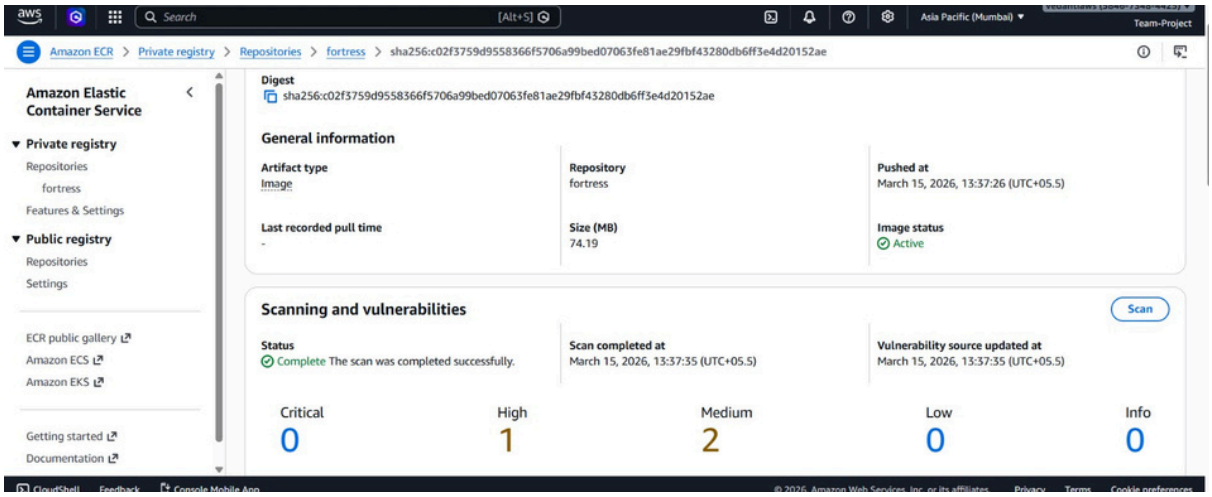
Trivy container vulnerability scan blocking insecure image in the CI pipeline.



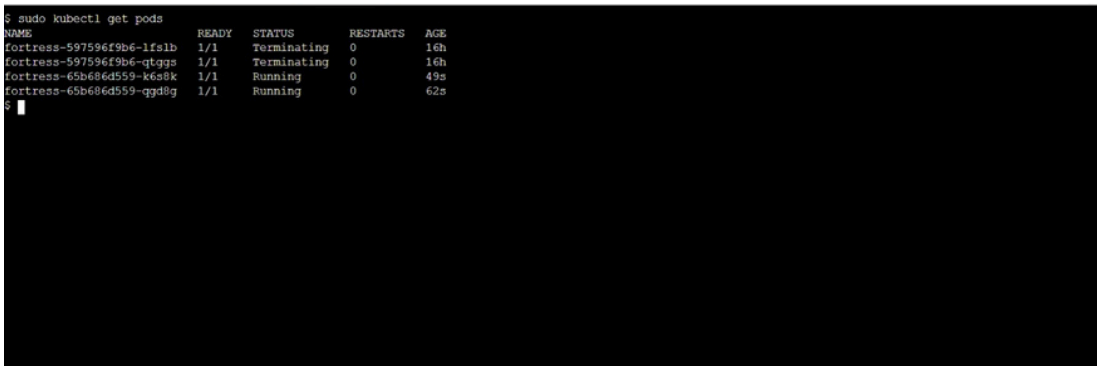
GitHub Actions Successful Pipeline -



Docker image successfully pushed to Amazon Elastic Container Registry (ECR). -

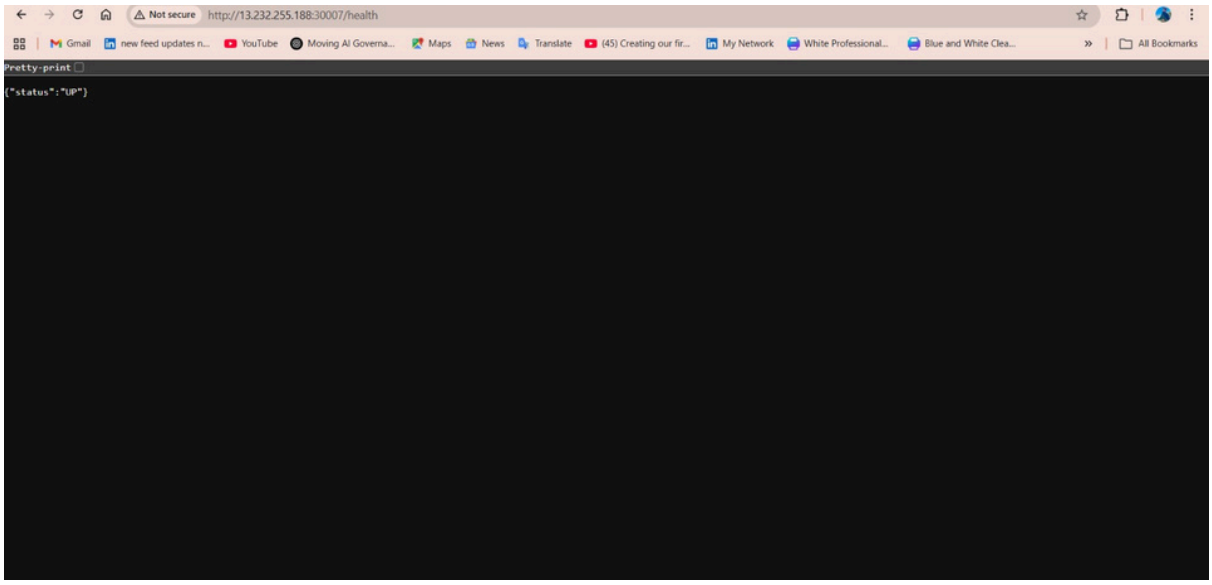


Kubernetes deployment running on the K3s cluster hosted on AWS EC2. -



```
$ sudo kubectl get pods
NAME                        READY   STATUS        RESTARTS   AGE
fortress-597596f9b6-lfslb   1/1     Terminating   0          16h
fortress-597596f9b6-qtggs   1/1     Terminating   0          16h
fortress-65b686d559-k6s8k   1/1     Running       0          49s
fortress-65b686d559-qgd8g   1/1     Running       0          62s
$
```

Application health endpoint successfully responding from Kubernetes service. -



Final Outcome of Project

- Secret scanning
- Dependency vulnerability scanning
- Container security scanning
- Docker image build
- Secure container registry (Amazon ECR)
- Kubernetes deployment on AWS EC2
- Automated CI/CD workflow
- Security gate preventing vulnerable deployments